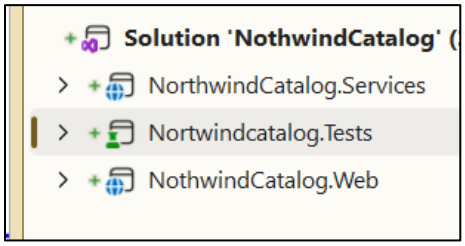
Northwind Catalog Project Documentation

1. Objective

Build a 3-layer .NET application using DB First approach, Web API, and MVC.

2. Solution Structure

• NorthwindCatalog.Web (MVC)
• NorthwindCatalog.Services (Business Logic)
• NorthwindCatalog.Tests (Unit Tests)



3. DB First Approach

Used EF Core scaffolding to generate models from Northwind database.

```
PS C:\Users\abhay\source\repos\NothwindCatalog\NorthwindCatalog.Services> dotnet ef dbcontext scaffold "
Server=.\SQLEXPRESS;Database=Northwind;Trusted_Connection=True;TrustServerCertificate=True;" Microsoft.E
ntityFrameworkCore.SqlServer -o Models
Build started...
Build succeeded.
To protect potentially sensitive information in your connection string, you should move it out of source
 code. You can avoid scaffolding the connection string by using the Name= syntax to read it from configu
ration - see https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing connection st
rings, see https://go.microsoft.com/fwlink/?LinkId=723263.
PS C:\Users\abhay\source\repos\NothwindCatalog\NorthwindCatalog.Services> dotnet ef dbcontext scaffold "
Server=.\SQLEXPRESS;Database=Northwind;Trusted_Connection=True;TrustServerCertificate=True;" Microsoft.E
ntityFrameworkCore.SqlServer -o Models --table Categories --table Products
Build started...
```

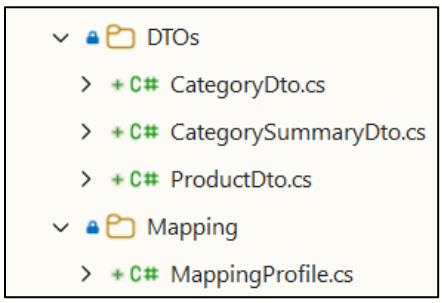
4. Fluent API Configuration

Configured relationships and column types using OnModelCreating.

```
modelBuilder.Entity<Product>(entity =>
{
    entity.HasKey(e => e.ProductId);

    entity.Property(e => e.ProductId)
        .HasColumnName("ProductID");

    entity.Property(e => e.ProductName)
        .HasMaxLength(40);

    entity.Property(e => e.QuantityPerUnit)
        .HasMaxLength(20);

    entity.Property(e => e.UnitPrice)
        .HasColumnType("decimal(10,2)");

    entity.Property(e => e.UnitsInStock)
        .HasDefaultValue((short)0);

    entity.Property(e => e.UnitsOnOrder)
        .HasDefaultValue((short)0);

    entity.Property(e => e.ReorderLevel)
        .HasDefaultValue((short)0);

    entity.Property(e => e.CategoryId)
        .HasColumnName("CategoryID");

    entity.Property(e => e.SupplierId)
        .HasColumnName("SupplierID");

    // Relationship
    entity.HasOne(d => d.Category)
        .WithMany(p => p.Products)
        .HasForeignKey(d => d.CategoryId)
        .HasConstraintName("FK_Products_Categories");
});
```

5. DTOs

Created CategoryDto, ProductDto, and CategorySummaryDto.
Included calculated field InventoryValue.



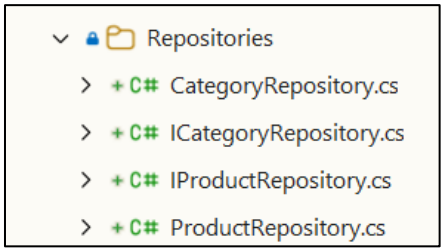
6. AutoMapper

Mapped entities to DTOs and handled ImageUrl transformation.

```
using AutoMapper;
using NorthwindCatalog.Services.Models;
using NorthwindCatalog.Services.DTOs;
namespace NorthwindCatalog.Services.Mapping
{

    2 references
    public class MappingProfile : Profile
    {
        0 references
        public MappingProfile()
        {
            CreateMap<Category, CategoryDto>()
      .ForMember(dest => dest.ImageUrl,
          opt => opt.MapFrom(src =>
              "/images/" +
              src.CategoryName
                  .Replace(" ", "")
                  .Replace("/", "")
              + ".jpeg"));
            CreateMap<Product, ProductDto>();
        }
    }
}
```

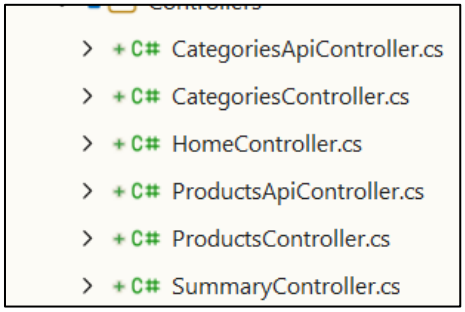
7. Repository Pattern

Implemented ICategoryRepository and IProductRepository with async methods.



8. Web API Layer

Created APIs for Categories, Products (by category), and Summary.



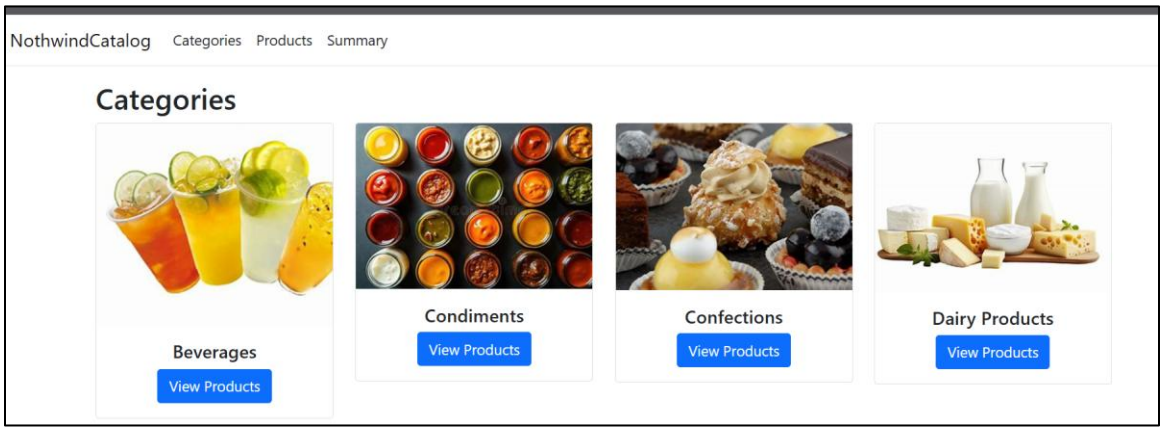
9. MVC Integration

Used HttpClient to consume APIs in MVC controllers.

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container
builder.Services.AddHttpClient("api", client =>
{
    client.BaseAddress = new Uri("https://localhost:7201/");
});
builder.Services.AddScoped<ICategoryRepository, CategoryRepository>();
builder.Services.AddScoped<IProductRepository, ProductRepository>();
builder.Services.AddControllersWithViews();
builder.Services.AddAutoMapper(typeof(MappingProfile));
// Add DbContext
builder.Services.AddDbContext<NorthwindContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();
```

10. UI Implementation

Designed Category cards and Product cards.
Implemented UI for category-wise products.



11. Summary Page

Displayed analytics: Product count, average price, most expensive product.

Category Summary

Category	Product Count	Avg Price	Most Expensive
Beverages	12	37.979166	Côte de Blaye
Condiments	12	23.062500	Vegie-spread
Confections	13	25.160000	Sir Rodney's Marmalade
Dairy Products	10	28.730000	Raclette Courdavault
Grains/Cereals	7	20.250000	Gnocchi di nonna Alice

12. Unit Testing

Used xUnit to test InventoryValue logic with multiple test cases.

```
namespace NorthwindCatalog.Tests
{
    0 references
    public class ProductTests
    {
        [Fact]
        | 0 references
        public void InventoryValue_Should_Return_Correct_Value()
        {
            // Arrange
            var product = new ProductDto
            {
                UnitPrice = 100,
                UnitsInStock = 5
            };

            // Act
            var result = product.InventoryValue;

            // Assert
            Assert.Equal(500, result);
        }
        [Fact]
        | 0 references
        public void InventoryValue_Should_Be_Zero_When_Stock_Is_Zero()
        {
            var product = new ProductDto
            {
                UnitPrice = 100,
                UnitsInStock = 0
            };

            var result = product.InventoryValue;

            Assert.Equal(0, result);
        }
        [Fact]
        | 0 references
        public void InventoryValue_Should_Be_Zero_When_Price_Is_Zero()
        {
            var product = new ProductDto
            {
                UnitPrice = 0,
                UnitsInStock = 10
            };

            var result = product.InventoryValue;
```

13. Conclusion

Successfully built a layered application with API integration, UI, and testing.